

Compte Rendu – TP2 : Manipulation du Système de Fichiers Distribué HDFS

1. Introduction

Ce TP a pour objectif de comprendre le fonctionnement du système de fichiers distribué HDFS (Hadoop Distributed File System) et de manipuler ses principales commandes. Nous avons appris à créer des répertoires, transférer des fichiers, lire et récupérer des données, et utiliser l'interface web d'HDFS.

2. Préparation de l'environnement

.Démarrage du service HDFS :

```
vboxuser@ubuntu:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [ubuntu]
```

.Vérification des services Hadoop :

```
vboxuser@ubuntu:~$ jps
24626 SecondaryNameNode
24290 NameNode
24409 DataNode
24765 Jps
```

.Accès à HDFS :

```
vboxuser@ubuntu:~$ hdfs dfs -ls /
```

3. Exercices pratiques

3.1. Création d'un répertoire utilisateur

Un dossier personnel **/user/tp2** est créé dans HDFS :

```
vboxuser@ubuntu:~$ hdfs dfs -mkdir -p /user/tp2
vboxuser@ubuntu:~$ hdfs dfs -ls /user
```

3.2. Transfert de fichiers vers HDFS

Création de deux fichiers texte localement :

```
vboxuser@ubuntu:~$ echo "big data est l'avenir de la donnée." > texte1.txt
vboxuser@ubuntu:~$ echo "hadoop est un framework open-source " > texte2.txt
```

Transfert vers HDFS :

```
vboxuser@ubuntu:~$ hdfs dfs -put texte1.txt /user/tp2/
vboxuser@ubuntu:~$ hdfs dfs -put texte2.txt /user/tp2/
vboxuser@ubuntu:~$ hdfs dfs -ls /user/tp2/
```

Vérification :

```
vboxuser@ubuntu:~$ hdfs dfs -ls /user/tp2
Found 2 items
-rw-r--r--   3 vboxuser supergroup      36 2025-11-17 20:51 /user/tp2/texte1.txt
-rw-r--r--   3 vboxuser supergroup      37 2025-11-17 20:57 /user/tp2/texte2.
```

Afficher le contenu d'un fichier dans HDFS :

```
vboxuser@ubuntu:~$ hdfs dfs -cat /user/tp2/texte1.txt
pig data est l'avenir de la donnee.
```

Télécharger un fichier depuis HDFS vers le système local :

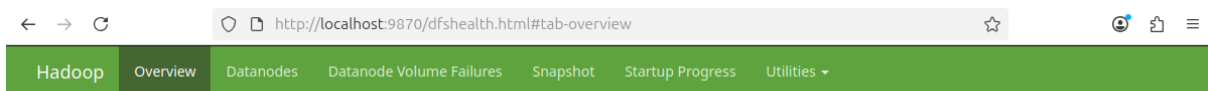
```
vboxuser@ubuntu:~$ hdfs dfs -get /user/tp2/texte2.txt
get: `texte2.txt': File exists
vboxuser@ubuntu:~$
```

3.4. Manipulation des répertoires

Création d'un sous-dossier et Déplacement d'un fichier :

```
vboxuser@ubuntu:~$ hdfs dfs -mkdir /user/tp2/docs
vboxuser@ubuntu:~$ hdfs dfs -mv /user/tp2/texte1.txt /user/tp2/docs/
vboxuser@ubuntu:~$
```

4. Visualisation via l'interface Web



Overview 'localhost:9000' (✓active)

Started:	Mon Nov 17 20:19:23 +0000 2025
Version:	3.4.2, r84e8b89ee2ebe6923691205b9e171badde7a495c
Compiled:	Wed Aug 20 10:30:00 +0000 2025 by ahmarsu from branch-3.4.2
Cluster ID:	CID-6e9486ee-9443-4042-9a66-b0ffd3174521
Block Pool ID:	BP-892696557-10.0.2.15-1763403815312

Summary

Security is off.

Safemode is off.

6 files and directories, 2 blocks (2 replicated blocks, 0 erasure coded block groups) = 8 total filesystem object(s).

Heap Memory used 81.01 MB of 317.5 MB Heap Memory. Max Heap Memory is 1.58 GB.

Non Heap Memory used 75.54 MB of 77.3 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	115.07 GB
----------------------	-----------

5. courte explication

a) Qu'est-ce que la réplication dans HDFS ?

La réplication est le mécanisme par lequel chaque bloc de données est copié sur plusieurs DataNodes.

Le facteur de réplication par défaut est 3.

Cela permet d'assurer :

- la tolérance aux pannes,
- la disponibilité des données,
- la continuité du service même si un nœud tombe en panne.

b) Pourquoi HDFS est-il plus fiable qu'un système de fichiers classique ?

HDFS est plus fiable grâce à plusieurs caractéristiques :

- **Réplication des blocs** → les données ne sont jamais perdues même si un DataNode tombe.
- **NameNode et SecondaryNameNode** gèrent les métadonnées et assurent la cohérence.
- **Architecture distribuée** → stockage sur plusieurs machines.
- **Tolérance aux pannes intégrée** → rééquilibrage automatique des blocs.

Un système local classique perd les données si le disque dur tombe en panne, contrairement à HDFS.

Conclusion

Ce TP nous a permis de comprendre la structure d'HDFS, son fonctionnement basé sur les blocs et la réplication, et de manipuler les principales commandes du système. La visualisation via l'interface web a aidé à mieux comprendre la répartition des données et la gestion des blocs. HDFS se révèle particulièrement adapté aux environnements Big Data nécessitant fiabilité, disponibilité et tolérance aux pannes.